Streams, Indie Devs and Nostalgia: What's Trending in Gaming on Bluesky?

COMP3020 Social Web Analytics: Group Project Report

Contents

1 Investigation overview **2**

2 Data collection **2**

3 Data cleaning and text representation **4**

4 RQ1: What is the gaming conversation about, and what is trending? **5**
4.1 What is trending? 8

5 RQ2: Can gaming posts be grouped into conversation themes? **9**

6 RQ3: Does the theme of a post affect its engagement? **15**
6.1 Test 1: Engagement versus theme (chi-squared test of independence) 15
6.2 Test 2: Do posts with images or video get more likes? (Welch two-sample t-test) 17

7 RQ4: Who connects the gaming conversation? (network analysis) **18**

8 Overall findings **25**

9 Limitations **26**

10 Reproducibility **26**

Group 22

Group member	Student ID	Contribution
Anamol Gautam	22226775	41%
Dikshant Thapa	22214022	40%
Ricky Melo	22090685	19%

GitHub repository: https://github.com/22090685/COMP3060-SocialWebAnalytics-Group-22

1 Investigation overview

Bluesky has grown into a major home for game developers, journalists and players. Our investigation asks one overall question: **what is the gaming community on Bluesky talking about, which kinds of gaming conversation get attention, and who connects that conversation?** We answer it in four linked stages. Each stage builds on the one before it:

Stage	Research question	Technique
RQ1	Which gaming topics dominate the conversation, and which are *trending* (rising) in the most recent week compared with the previous three?	Text analysis, visualisation, two-proportion tests
RQ2	Can posts be grouped into distinct conversation **themes**?	k-means clustering of TF-IDF vectors (cosine geometry), checked with hierarchical clustering
RQ3	Is engagement (getting at least one like) **independent of theme**, and do posts with images or video get more likes?	Chi-squared test of independence; Welch two-sample t-test
RQ4	Who are the **central accounts** in the gaming interaction network, and do people mostly interact *within* their own theme?	Directed reply/mention/quote network, centrality, assortativity

RQ1 shows *what* is being discussed. RQ2 turns those words into themes. RQ3 uses the themes to test whether some kinds of gaming content are rewarded more than others. RQ4 asks which accounts sit at the centre of the interactions and whether the themes found in RQ2 also separate the network.

```
library(dplyr); library(tidyr); library(stringr); library(lubridate)
library(ggplot2); library(scales); library(tm); library(wordcloud)
library(RColorBrewer); library(cluster); library(igraph); library(knitr)
set.seed(3020)                          # all random steps are reproducible
dir.create("figures", showWarnings = FALSE)
theme_set(theme_minimal(base_size = 11))
```

2 Data collection

Source and access. Data were collected from Bluesky through the official AT Protocol API with the script `R/01_collect_bluesky.R` (in the repository). The script authenticates with an *app password* (`com.atproto.server.createSession`) and calls the `app.bsky.feed.searchPosts` endpoint. Packages: `httr` (HTTP requests), `jsonlite` (JSON parsing), `dplyr`/`tidyr` (tidying).

Sampling design. We used six deliberately *generic* search terms (`gaming`, `#gaming`, `videogames`, `#videogames`, `"video games"`, `gamer`). We avoided platform or game names so that the data would not decide in advance which games or companies "win". Results were restricted to English (`lang = "en"`) and sorted by `latest`. Search results cannot be paged indefinitely, so each day of the 28-day window was split into four 6-hour blocks and up to 100 posts were taken from each block for each term. This spreads the sample evenly over days and times of day, instead of returning only the newest posts. Posts matched by several terms were kept once. For every account appearing in the data (authors, and accounts that were replied to, mentioned or quoted), the profile was retrieved with `app.bsky.actor.getProfiles` (follower counts).

```
posts_raw <- readRDS("data/posts.rds")
profiles  <- read.csv("data/profiles.csv", stringsAsFactors = FALSE)
clog      <- read.csv("data/collection_log.csv", stringsAsFactors = FALSE)
win       <- as.POSIXct(attr(posts_raw, "window"), format = "%Y-%m-%dT%H:%M:%SZ", tz = "UTC")
collected <- attr(posts_raw, "collected_at")
c(posts = nrow(posts_raw), variables = ncol(posts_raw), accounts = n_distinct(posts_raw$author_did),
  api_calls = nrow(clog))
```

```
#>     posts variables  accounts api_calls
#>     46254        22     22000       672
```

Table 2: Main variables collected for each post.

Variable	Description
uri	Unique post identifier
author_did / author_handle	Author's permanent ID and handle
created_at	Time the post was written (UTC)
text	Post text (max 300 characters)
like/repost/reply/quote_count	Engagement at the time of collection
is_reply, reply_to_did	Whether the post is a reply, and to whom (taken from the parent post's URI)
mention_dids, quoted_did	Accounts mentioned / quoted in the post
embed_type, has_media, has_link	Attached images/video, link cards, quoted posts
hashtags	Hashtags used
followers (profiles.csv)	Author follower count at collection

The collection window was 17 Sep 2026 to 14 Oct 2026 (UTC), and data were collected on 15 Oct 2026. The API returned **46,254 unique posts** by **22,000 accounts**.

Limitations of the collection. (i) Keyword search only finds posts that use our terms. Gaming posts that never say "game"/"gaming" (e.g. "Silksong is out!") are missed, so the sample is *posts that explicitly frame themselves as gaming*. (ii) Each 6-hour block is capped at 100 posts per term. Busy blocks are therefore under-sampled, and our counts are a *sample*, not the full population (the API's `hitsTotal` gives a rough population size). (iii) Engagement counts are a snapshot at collection time, so very recent posts have had less time to collect likes. (iv) Bluesky's language tags are set by users' apps and are not always correct. (v) Only public posts are visible. Deleted posts and blocked accounts are missing.

```r
vol <- clog %>% mutate(day = as.Date(substr(since, 1, 10))) %>%
  filter(query == "gaming") %>% group_by(day) %>%
  summarise(estimated_matches = sum(hits_total, na.rm = TRUE), sampled = sum(n_returned))
ggplot(vol, aes(day)) +
  geom_col(aes(y = estimated_matches), fill = "grey75") +
  geom_line(aes(y = sampled), colour = "#D55E00", linewidth = 1) +
  labs(x = NULL, y = "Posts per day", subtitle = "Bars: estimated matches for 'gaming'; line: posts sampled for
  ↪  'gaming'")
```

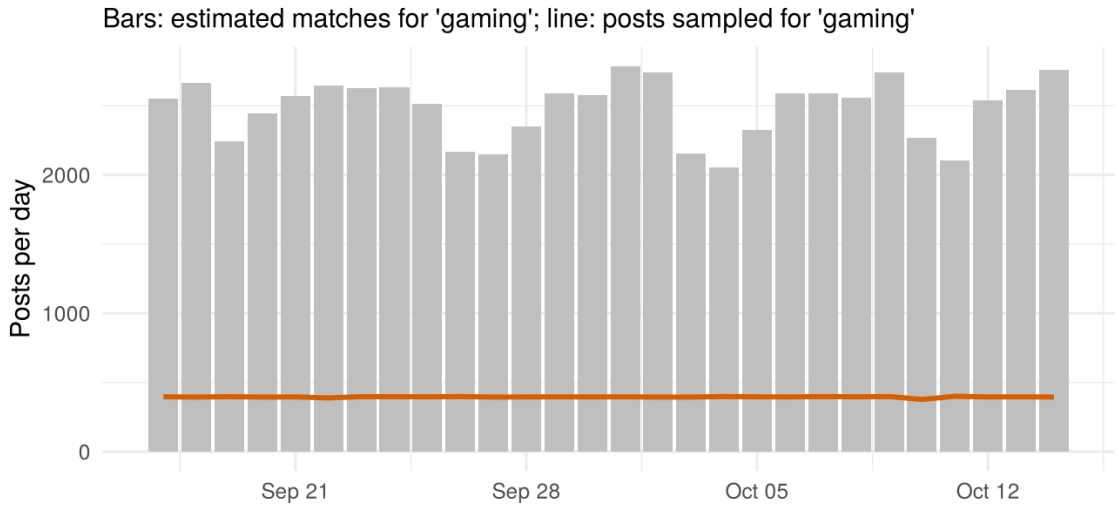


Figure 1: Estimated daily volume of posts matching 'gaming' (API hitsTotal) compared with the posts actually sampled per day.

3 Data cleaning and text representation

```
posts <- posts_raw %>%
  mutate(created = ymd_hms(created_at, quiet = TRUE),
         followers = profiles$followers[match(author_did, profiles$did)],
         has_media = embed_type %in% c("app.bsky.embed.images", "app.bsky.embed.video",
                                       "app.bsky.embed.gallery", "app.bsky.embed.recordWithMedia")) %>%
  filter(!is.na(created), created >= win[1], created < win[2])   # drop back-dated / bad timestamps

JOIN_EXCLUDE <- paste(c("top", "day", "days", "the", "and", "for", "part", "episode", "chapter", "season",
  "week", "weeks", "year", "years", "level", "act", "vol", "page", "over", "about", "with", "from", "only",
  "than", "just", "last", "first", "under", "all", "get", "got", "has", "had", "was", "are", "new", "now",
  "out", "off", "one", "two", "into", "after", "before", "hours", "hour", "min", "minutes", "times",
  "number", "rank", "round", "stage", "wave", "floor", "run", "out", "pick", "lot"), collapse = "|")
clean_text <- function(x) {
  x <- stringi::stri_trans_general(x, "Latin-ASCII")             # Pokémon -> Pokemon
  x <- tolower(x)
  x <- gsub("https?://\\S+|www\\.\\S+", " ", x)                  # URLs
  x <- gsub("@[a-z0-9._-]+", " ", x)                             # @handles
  x <- gsub("\\S*[a-z0-9-]+\\.[a-z]{2,}/\\S*", " ", x)                 # shortened links: site.com/path...
  x <- gsub("\\b[a-z0-9-]+\\.(com|net|org|gg|tv|io|social|app|co)\\b", " ", x)  # bare domains
  x <- gsub("'", "", x)                                          # don't -> dont
  x <- gsub("#", " ", x)                                         # keep the hashtag word
  x <- gsub("[^a-z0-9 ]", " ", x)                                # emoji, punctuation
  # join a name to a following 1-2 digit number (switch 2 -> switch2, gta 6 -> gta6, battlefield 6 ->
  # battlefield6) unless the word is a common word such as "top 10", "day 1" or "the 3"
  x <- gsub(paste0("\\b(?!(?:", JOIN_EXCLUDE, ")\\b)([a-z]{3,}) ([0-9]{1,2})\\b"), "\\1\\2", x, perl = TRUE)
  x <- gsub("\\b[0-9]+\\b", " ", x)                              # stand-alone numbers
  gsub("\\s+", " ", trimws(x))
}
posts$clean <- clean_text(posts$text)
```

Stop words. Besides the standard English list, we use the larger **SMART** stop-word list supplied with `tm`. We also remove (a) our own **search terms** and their variants, because by construction they appear in almost every post and carry no information about *what* is discussed; (b) generic words that are very common in short social posts but say nothing about a topic ("love", "first", "never", "feel", "life", "check"); and (c) leftover fragments of contractions and a few very frequent Spanish/Portuguese function words ("de", "que"), which appear because some users' apps tag non-English posts as English. Our first clustering attempt used only the standard list. Generic words then dominated the similarity between posts, and the result was a single uninformative cluster, so this stronger list is a deliberate analytical decision. We did **not** remove platform, studio or game names, because those are what we want to find.

```
generic <- c("game", "games", "gaming", "gamer", "gamers", "gamergirl", "video", "videos", "videogame",
  "videogames", "videogaming", "play", "plays", "playing", "played", "player", "players",
  "just", "get", "got", "getting", "like", "one", "now", "can", "will", "really", "also", "even", "still",
  "much", "lot", "lots", "way", "thing", "things", "make", "made", "know", "think", "going", "want", "see",
  "say", "said", "back", "good", "great", "time", "day", "today", "people", "well", "love", "first", "new",
  "never", "need", "go", "something", "feel", "feels", "life", "ever", "best", "better", "fun", "little",
  "many", "every", "always", "actually", "maybe", "right", "left", "probably", "years", "year", "ago", "old",
  "real", "long", "look", "looking", "check", "watch", "work", "stuff", "help", "perfect", "favorite",
  "favourite", "god", "bad", "shit", "fuck", "fucking", "big", "last", "next", "week", "night", "start",
  "started", "finally", "post", "posts", "thank", "thanks", "pretty", "hard", "bit", "kind", "sure", "gets",
  "makes", "man", "guy", "guys", "person", "literally", "totally", "yet", "since", "around", "another",
  "everyone", "someone", "anyone", "anything", "everything", "nothing", "part", "whole", "full", "use",
  "using", "used", "try", "trying", "tried", "take", "takes", "find", "found", "keep", "put", "give",
  "read", "seen", "saw", "enough", "without", "though", "far", "home", "friends", "friend", "kids", "world",
  "times", "hours", "nice", "cool", "awesome", "amazing", "wait", "hope", "wish", "told", "tell", "talk",
  "talking", "idea", "point", "fact", "end", "least", "less", "else", "ones", "place", "done", "coming",
  "high", "top", "link", "amp", "im", "ive", "id", "dont", "doesnt", "didnt", "cant", "wont", "isnt",
  "thats", "youre", "theyre", "theres", "whats", "ll", "ve", "re", "th", "st", "nd", "rd", "oh", "ok",
  "lol", "u", "ur", "bsky", "social", "via", "yes", "yeah", "gonna", "gotta", "let", "lets", "us",
  "de", "la", "que", "el", "en", "es", "le", "un", "una", "pra", "para", "com", "um", "fyp", "ad")
my_stop <- unique(c(stopwords("english"), stopwords("SMART"), generic,
                    gsub("'", "", c(stopwords("english"), stopwords("SMART")))))
length(my_stop)
```

```
#> [1] 758
```

Removing unsuitable records. We removed (1) posts with **identical cleaned text**, which are mostly bots, spam and cross-posts; (2) posts with **fewer than 3 informative words** left after cleaning, which have no topical content to analyse or cluster; and (3) we capped each account at **`MAX_PER_AUTHOR` posts**, randomly chosen, so that a handful of automated accounts (deal bots, "now live" stream announcers) cannot single-handedly define the "topics".

```
MAX_PER_AUTHOR <- 30
count_info <- function(x) sapply(strsplit(x, " "), function(w) sum(!(w %in% my_stop) & nchar(w) >= 2))
steps <- c(`In collection window` = nrow(posts))
tx <- posts %>% distinct(clean, .keep_all = TRUE);           steps["After removing duplicate text"] <- nrow(tx)
tx <- tx %>% mutate(n_info = count_info(clean)) %>% filter(n_info >= 3)
steps["After removing posts with < 3 informative words"] <- nrow(tx)
tx <- tx %>% group_by(author_did) %>% slice_sample(n = MAX_PER_AUTHOR) %>% ungroup()
steps[paste("After capping at", MAX_PER_AUTHOR, "posts per account")] <- nrow(tx)
kable(data.frame(Step = names(steps), Posts = format(steps, big.mark = ",")), row.names = FALSE,
      caption = "Data cleaning steps.", booktabs = TRUE)
```

Table 3: Data cleaning steps.

Step	Posts
In collection window	46,253
After removing duplicate text	43,237
After removing posts with $<$ 3 informative words	39,743
After capping at 30 posts per account	34,627

Representation. Each post is a document. We build a **document-term matrix** (DTM) with `tm`, then weight it with **TF-IDF**, which gives a high weight to words that are frequent in a post but rare across posts. This makes distinctive topic words (e.g. a game title) stand out, and makes ubiquitous words matter less. We did not stem words, so that game titles and platform names stay readable.

```
corpus <- VCorpus(VectorSource(tx$clean))
dtm <- DocumentTermMatrix(corpus, control = list(stopwords = my_stop, wordLengths = c(2, 25)))
dtm
```

```
#> <<DocumentTermMatrix (documents: 34627, terms: 52274)>>
#> Non-/sparse entries: 381158/1809710640
#> Sparsity           : 100%
#> Maximal term length: 25
#> Weighting          : term frequency (tf)
```

4 RQ1: What is the gaming conversation about, and what is trending?

```
doc_freq <- slam::col_sums(dtm > 0)                       # in how many posts each word appears
top_words <- sort(doc_freq, decreasing = TRUE)[1:25]
fw <- data.frame(word = names(top_words), posts = as.numeric(top_words))
p_freq <- ggplot(fw, aes(reorder(word, posts), posts)) + geom_col(fill = "#0072B2") + coord_flip() +
  labs(x = NULL, y = "Number of posts containing word")
p_freq
```

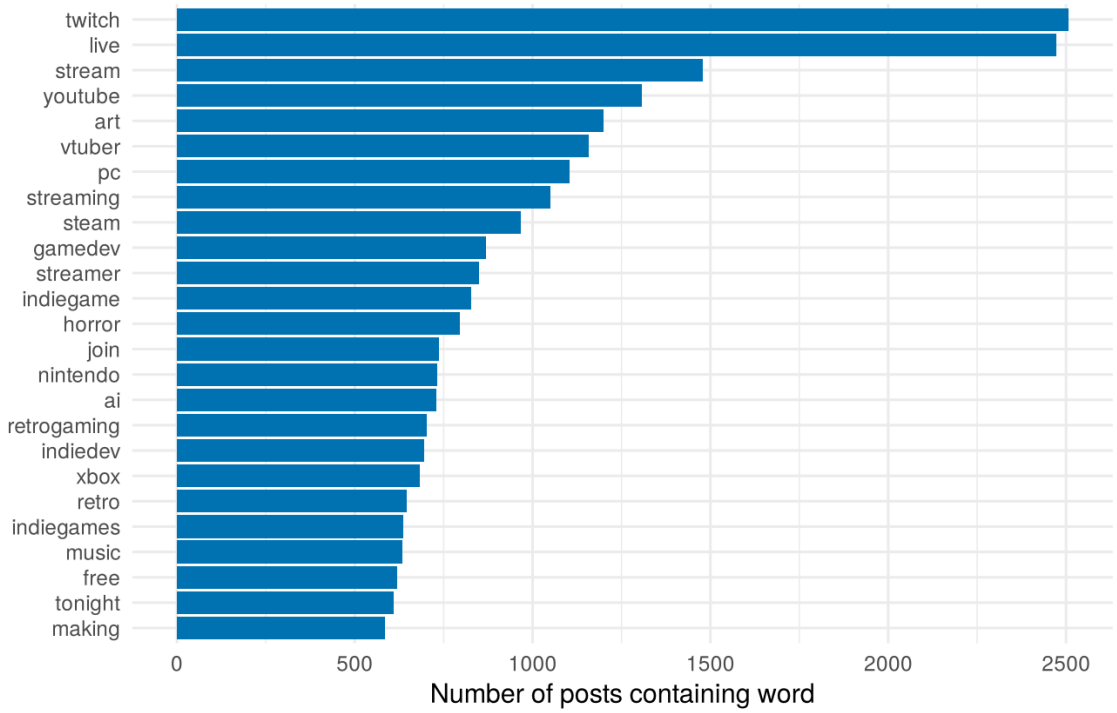


Figure 2: The 25 most frequent words (number of posts containing each word).

```
ggsave("figures/fig_top_words.png", p_freq, width = 6, height = 5, dpi = 300)
```

```
tfidf <- weightTfIdf(dtm)
tfidf_tot <- sort(slam::col_sums(tfidf), decreasing = TRUE)
pal <- brewer.pal(8, "Dark2")
wordcloud(names(tfidf_tot)[1:120], tfidf_tot[1:120], scale = c(3.5, 0.5), colors = pal,
          random.order = FALSE, rot.per = 0.15)
```



Figure 3: Word cloud weighted by total TF-IDF. Larger words are more distinctive across posts.

```r
png("figures/fig_wordcloud.png", width = 1800, height = 1800, res = 300)
par(mar = c(0, 0, 0, 0))
wordcloud(names(tfidf_tot)[1:120], tfidf_tot[1:120], scale = c(3.5, 0.5), colors = pal,
          random.order = FALSE, rot.per = 0.15); invisible(dev.off())
```

Single words can split names apart ("silk" and "song", "mario" and "kart"), so we also count **bigrams** (adjacent word pairs, after removing stop words) and **hashtags**.

```r
toks <- lapply(strsplit(tx$clean, " "), function(w) w[!(w %in% my_stop) & nchar(w) >= 2])
bigr <- unlist(lapply(toks, function(w) if (length(w) > 1) unique(paste(head(w, -1), tail(w, -1)))))
top_bi <- head(sort(table(bigr), decreasing = TRUE), 15)
tags <- unlist(strsplit(tx$hashtags[tx$hashtags != ""], ";"))
tags <- tags[!(tags %in% c("gaming", "videogames", "games", "videogame", "game", "gamer", "gamers"))]
top_tags <- head(sort(table(tags), decreasing = TRUE), 15)
bt <- data.frame(term = names(top_bi), n = as.numeric(top_bi), type = "Bigrams")
if (length(top_tags) > 0) bt <- rbind(bt, data.frame(term = paste0("#", names(top_tags)),
                                      n = as.numeric(top_tags), type = "Hashtags (excl. search tags)"))
ggplot(bt, aes(reorder(term, n), n)) + geom_col(fill = "#009E73") + coord_flip() +
  facet_wrap(~type, scales = "free") + labs(x = NULL, y = "Number of posts")
```

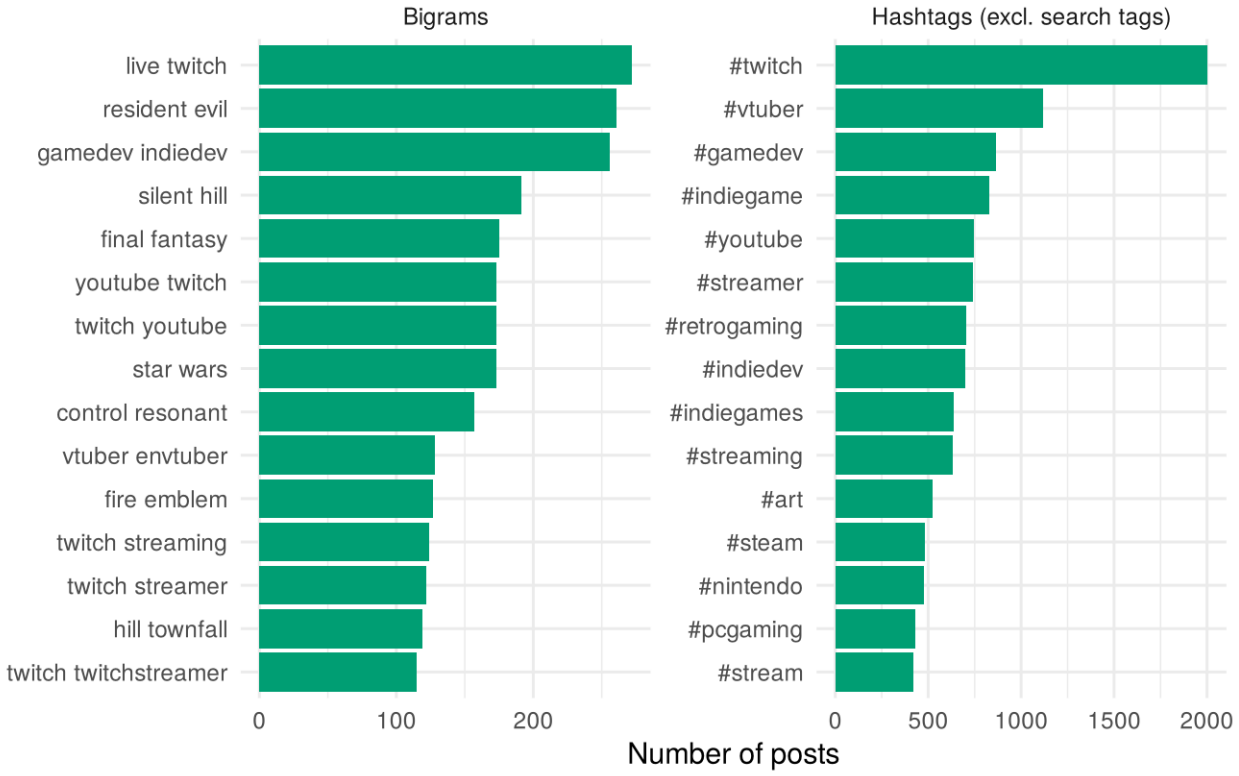


Figure 4: Most frequent bigrams (left) and hashtags (right).

Interpretation (RQ1, frequency). With the search terms and generic words removed, the most common words show that gaming Bluesky is dominated by **creators promoting their own work**. The top words (*twitch, live, stream, youtube, vtuber, streaming, streamer*) and the top hashtags (#twitch, #vtuber, #streamer) are all about live-streaming. The second block comes from **independent developers** (*gamedev, indiegame, indiedev, steam*). After that come **platforms** (*pc, steam, nintendo, xbox*), **retro gaming**, **art** and **AI**. Bigrams name the franchises that people discuss: *Resident Evil*, *Silent Hill* (including *Silent Hill Townfall*), *Final Fantasy*, *Star Wars*, *Fire Emblem* and *Control Resonant*. Horror is prominent because the window covers the start of October, the Halloween season. So the frequency analysis suggests that gaming Bluesky works less like a forum about blockbuster releases and more like a **showcase where streamers and indie developers advertise**, with fan discussion concentrated on a handful of franchises. RQ2 tests whether these impressions form distinct themes.

4.1 What is trending?

A word is **trending** if a larger share of **active accounts** used it in the **latest week** than in the **previous three weeks**. We count *accounts*, not posts. Our first attempt counted posts, and the "top trending" words (`minesweeper`, `flagged`, `cells`) all came from a single bot posting game results many times. Counting each account at most once per period stops one prolific account from manufacturing a trend. It also makes the observations closer to independent, which the test assumes. For every word appearing in at least 30 posts, we compare the two proportions with a **two-proportion test** (`prop.test`):

- $H_0 : p_{\text{latest week}} = p_{\text{earlier}}$ (the share of active accounts using the word did not change)
- $H_1 : p_{\text{latest week}} \neq p_{\text{earlier}}$

Because we test hundreds of words at once, some would look "significant" by chance alone. We therefore apply a **Bonferroni correction** (multiply each p-value by the number of tests) before calling a word significantly trending.

```
recent <- tx$created >= (win[2] - 7 * 86400)
cand <- names(doc_freq[doc_freq >= 30])
bin <- as.matrix(dtm[, cand] > 0) * 1
# collapse posts to one row per (account, period): did this account use the word in this period?
grp <- paste(tx$author_did, recent)
acc <- rowsum(bin, grp) > 0
acc_recent <- grepl("TRUE$", rownames(acc))
n_r <- sum(acc_recent); n_e <- sum(!acc_recent)              # active accounts in each period
x_r <- colSums(acc[acc_recent, , drop = FALSE]); x_e <- colSums(acc[!acc_recent, , drop = FALSE])
pv <- mapply(function(a, b) suppressWarnings(prop.test(c(a, b), c(n_r, n_e))$p.value), x_r, x_e)
trend <- data.frame(word = cand, recent = x_r, earlier = x_e,
                    share_recent = x_r / n_r, share_earlier = x_e / n_e,
                    log2_ratio = log2(((x_r + 0.5) / n_r) / ((x_e + 0.5) / n_e)),
                    p_adj = p.adjust(pv, method = "bonferroni")) %>% arrange(desc(log2_ratio))
sig <- trend %>% filter(p_adj < 0.05)
show <- bind_rows(head(sig, 12), tail(sig, 6)) %>% distinct(word, .keep_all = TRUE)
p_trend <- ggplot(show, aes(reorder(word, log2_ratio), log2_ratio, fill = log2_ratio > 0)) + geom_col() +
  coord_flip() + scale_fill_manual(values = c(`TRUE` = "#D55E00", `FALSE` = "#0072B2"), guide = "none") +
  labs(x = NULL, y = "log2(share of accounts in latest week / previous 3 weeks)")
p_trend
```

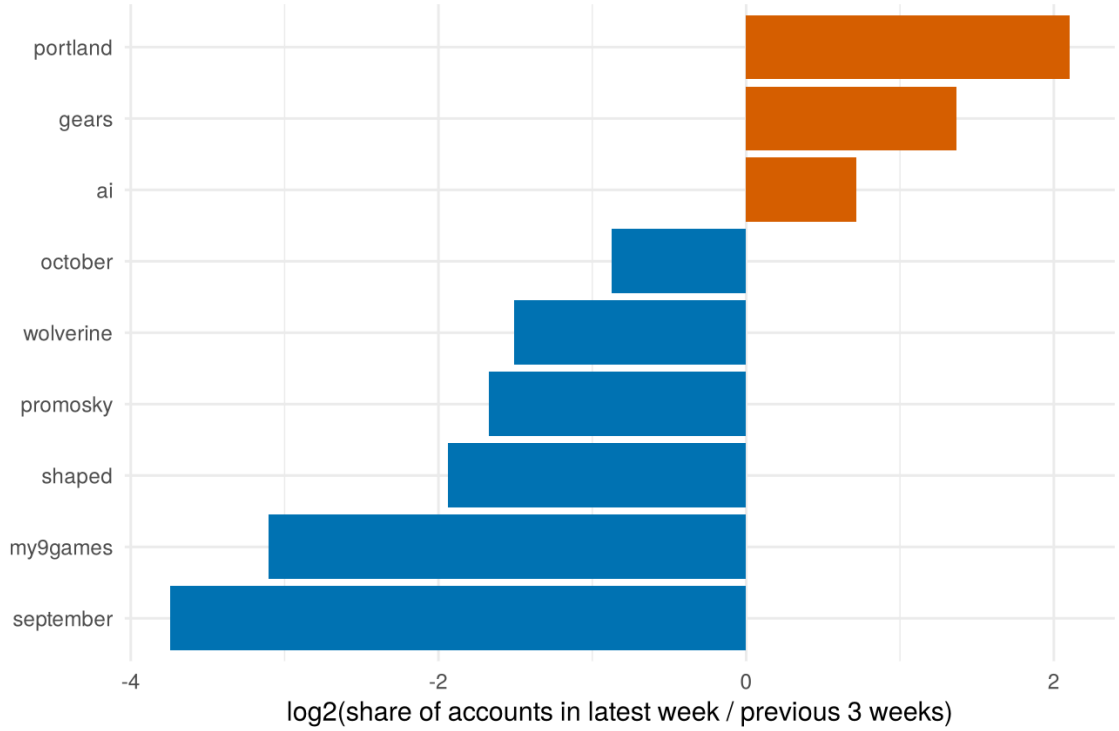


Figure 5: Words with the biggest rise (red) or fall (blue) in the share of active accounts using them in the latest week. All shown are significant after Bonferroni correction.

```r
ggsave("figures/fig_trending.png", p_trend, width = 6, height = 5, dpi = 300)
c(active_accounts_latest_week = n_r, active_accounts_earlier = n_e,
  words_tested = length(cand), significant_after_bonferroni = nrow(sig),
  rising = sum(sig$log2_ratio > 0), falling = sum(sig$log2_ratio < 0))
```

```
#>  active_accounts_latest_week      active_accounts_earlier                 words_tested
#>                         5933                        15298                         2457
#> significant_after_bonferroni                       rising                      falling
#>                            9                            3                            6
```

```r
kable(sig %>% transmute(word, `accounts (latest wk)` = recent, `accounts (earlier 3 wks)` = earlier,
      `share latest` = percent(share_recent, 0.01), `share earlier` = percent(share_earlier, 0.01),
      `Bonferroni p` = signif(p_adj, 2)), caption = "All words with a significant change in usage.",
      booktabs = TRUE, row.names = FALSE)
```

Table 4: All words with a significant change in usage.

word	accounts (latest wk)	accounts (earlier 3 wks)	share latest	share earlier	Bonferroni p
portland	27	16	0.46%	0.10%	2.0e-03
gears	55	55	0.93%	0.36%	1.0e-03
ai	230	361	3.88%	2.36%	5.4e-06
october	60	285	1.01%	1.86%	3.4e-02
wolverine	17	128	0.29%	0.84%	4.7e-02
promosky	23	193	0.39%	1.26%	4.7e-05
shaped	11	113	0.19%	0.74%	8.3e-03
my9games	3	77	0.05%	0.50%	6.2e-03
september	2	86	0.03%	0.56%	3.6e-04

Interpretation (RQ1, trending). Only 9 of 2457 words changed significantly after the Bonferroni correction. Week to week, the conversation is broadly **stable**, and the changes that do occur are driven by **events**. Reading the posts behind each word explains them:

- **Rising:** *portland* comes from the **Portland Retro Gaming Expo** (PRGE), which people attended and posted about in the final week. *gears* comes from the release of ***Gears of War: E-Day***, with posts announcing its launch on PC and Xbox and people streaming its campaign. *ai* rose from 2.4% to 3.9% of active accounts, reflecting debate about AI tools in games and the games industry.
- **Falling:** *my9games* and *shaped* belong to the viral **"#my9games: the 9 games that shaped me"** meme, which peaked in the earlier weeks and then faded. Posts about ***Marvel's Wolverine*** (playthroughs, reviews and disappointment) also faded. *promosky* is the hashtag of a promotional account.
- **Sanity check:** *september* almost disappeared in the latest week (2 accounts, against 86 earlier), as it should. *october* also fell, because "it's October!" posts cluster at the start of the month. Calendar words behaving as expected gives us confidence that the method detects real changes.

5 RQ2: Can gaming posts be grouped into conversation themes?

Representation and distance. We cluster posts using their TF-IDF vectors, restricted to the words used in at least 0.3% of posts (about 100 posts), because rarer words make vectors sparse and noisy. We keep only posts containing **at least 3** of these words. A post with one or two common words cannot be placed reliably in any theme. Text similarity should not depend on post length, so we use **cosine distance**. We normalise each post vector to length 1, and then Euclidean distance becomes a monotone function of cosine distance ($\|a-b\|^2 = 2(1-\cos\theta)$). This lets us use **k-means** (which minimises within-cluster squared Euclidean distance) as a cosine-based clustering method. Posts with no remaining words are dropped.

```r
dtm_c <- removeSparseTerms(dtm, 0.997)
keep <- slam::row_sums(dtm_c > 0) >= 3                    # posts with >= 3 clustering words
X <- as.matrix(weightTfIdf(dtm_c[keep, ]))
```

```
X <- X / sqrt(rowSums(X^2))                                    # unit-length rows -> cosine geometry
c(posts_clustered = nrow(X), terms = ncol(X))
```

```
#> posts_clustered           terms
#>           21083             559
```

Choosing the number of clusters. We ran k-means for $k = 2, \ldots, 14$ (5 random starts each, keeping the best solution; the final model uses 20 starts). We look at the **elbow** in total within-cluster sum of squares (SSW) and at the average **silhouette width**, computed on a random sample of 2,000 posts using cosine distance.

```
ks <- 2:14
samp <- sample(nrow(X), min(2000, nrow(X)))
D_s <- dist(X[samp, ])                                         # Euclidean on unit vectors ~ cosine
fits <- lapply(ks, function(k) kmeans(X, centers = k, nstart = 5, iter.max = 100))
ssw <- sapply(fits, function(f) f$tot.withinss)
sil <- sapply(fits, function(f) mean(silhouette(f$cluster[samp], D_s)[, 3]))
kd <- data.frame(k = ks, SSW = ssw, silhouette = sil) %>% pivot_longer(-k)
ggplot(kd, aes(k, value)) + geom_line() + geom_point() + facet_wrap(~name, scales = "free_y") +
  scale_x_continuous(breaks = ks) + labs(y = NULL)
```

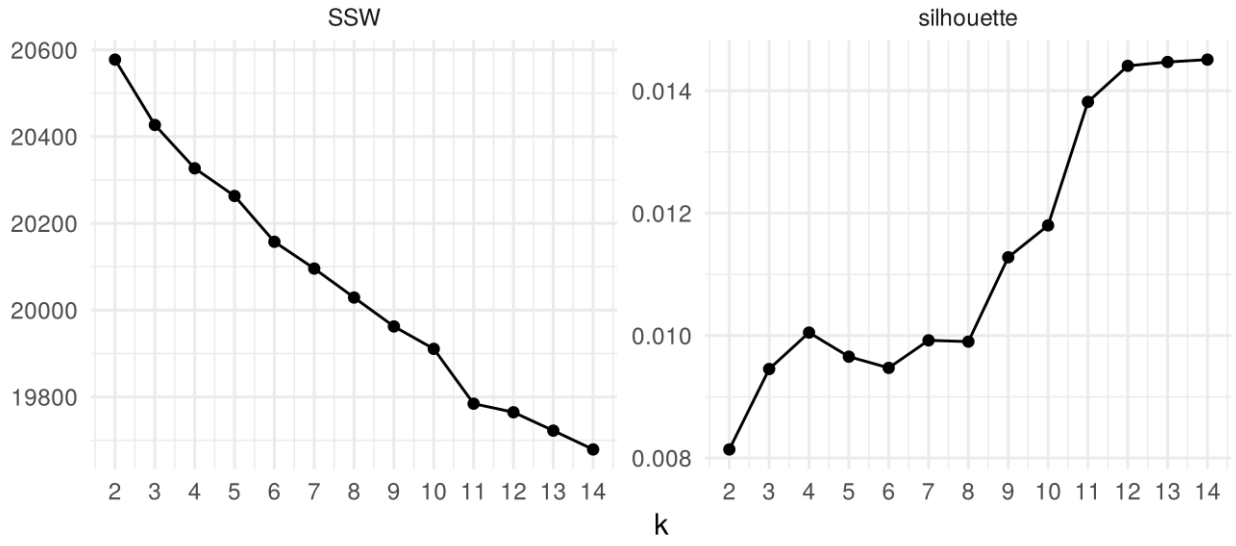


Figure 6: Elbow (left) and average silhouette width (right) for k = 2..14.

```
K <- 9                    # chosen after inspecting the plots and the clusters for k = 6..14 (see text)
set.seed(3020)
km <- kmeans(X, centers = K, nstart = 20, iter.max = 100)    # final model: more random starts
tx_c <- tx[keep, ]; tx_c$cluster <- km$cluster
K
```

```
#> [1] 9
```

Choice of k. SSW decreases steadily with no clear elbow. The average silhouette width is very low for every k (0.008 to 0.015), which means posts overlap heavily. This is typical of very short texts that share few words. Neither statistic identifies a "natural" k, so we combined them with **interpretability**. We inspected the clusters for $k = 6, \ldots, 14$. The silhouette shows its first clear rise at $k = 9$. For smaller k, themes such as horror were not consistently separated from the general cluster. For larger k, the extra clusters mostly split the streaming posts into further near-duplicate groups or created very small single-topic clusters. We therefore chose $k = 9$, the smallest k that separates all the substantive themes.

What is in each cluster? For each cluster we list the words with the highest mean TF-IDF weight in the cluster centroid, and we give each cluster a short theme name based on those words and on reading a sample of its posts.

```
top_terms <- apply(km$centers, 1, function(v) paste(names(sort(v, decreasing = TRUE))[1:8], collapse = ", "))
```

```
# Theme names were written after reading each cluster's top terms and example posts. To make the
# labels robust to the arbitrary numbering of k-means clusters, each name is attached to the cluster
# whose centroid gives its keywords the most weight; the largest, least specific cluster is "General".
label_rules <- list(
  "VTubers"          = c("vtuber", "envtuber", "pngtuber", "vsky"),
  "Horror"           = c("horror", "resident", "evil", "silent", "hill"),
  "Nintendo & retro" = c("nintendo", "retrogaming", "retro", "mario", "zelda", "switch2"),
  "PC & consoles"    = c("pc", "steam", "xbox", "ps5", "pcgaming", "playstation"),
  "Go-live invites"  = c("live", "hang", "livestream", "join"),
  "Indie dev"        = c("gamedev", "indiedev", "indiegame", "wishlist", "pixelart"),
  "Streamer promo"   = c("streaming", "streamer", "twitchstreamer", "discord", "stream"),
  "AI & tech news"   = c("ai", "news", "technology", "business"))
theme_names <- rep(NA_character_, K)
theme_names[which.max(km$size)] <- "General / mixed"
score <- sapply(label_rules, function(w) rowSums(km$centers[, intersect(w, colnames(km$centers)), drop = FALSE]))
score[which.max(km$size), ] <- -Inf
while (any(is.finite(score)) && any(is.na(theme_names))) {           # greedy one-to-one matching
  ij <- which(score == max(score), arr.ind = TRUE)[1, ]
  theme_names[ij[1]] <- colnames(score)[ij[2]]; score[ij[1], ] <- -Inf; score[, ij[2]] <- -Inf
}
left <- which(is.na(theme_names))                                     # any unmatched cluster
theme_names[left] <- top_terms[left]
ord <- order(theme_names == "General / mixed", -km$size)              # order levels by size, General last
tx_c$theme <- factor(theme_names[tx_c$cluster], levels = theme_names[ord])
theme_cols <- c("#1B9E77", "#D95F02", "#7570B3", "#E7298A", "#66A61E", "#E6AB02", "#A6761D", "#1F78B4",
                "#B2182B", "#35978F", "#8C510A", "#C51B7D")                   # no greys: grey = General
pal_k <- setNames(theme_cols[seq_len(K - 1)], setdiff(levels(tx_c$theme), "General / mixed"))
pal_k <- c(pal_k, `General / mixed` = "grey65")
csum <- tx_c %>% group_by(theme) %>%
  summarise(posts = n(), share = percent(n() / nrow(tx_c), 0.1), `top centroid terms` = top_terms[cluster[1]])
kable(csum, caption = "Themes (clusters): sizes and top centroid terms.", booktabs = TRUE) %>%
  kableExtra::column_spec(4, width = "10cm") %>% kableExtra::kable_styling(latex_options = "hold_position", font_size
↪  = 8)
```

Table 5: Themes (clusters): sizes and top centroid terms.

theme	posts	share	top centroid terms
Streamer promo	1554	7.4%	twitch, stream, streaming, streamer, youtube, tonight, twitchstreamer, join
Go-live invites	1537	7.3%	live, twitch, stream, hang, youtube, join, livestream, streamer
PC & consoles	1456	6.9%	pc, steam, xbox, ps5, sale, pcgaming, playstation, halo
Nintendo & retro	1140	5.4%	nintendo, retrogaming, retro, retrogames, switch2, playstation, mario, switch
VTubers	928	4.4%	vtuber, twitch, envtuber, live, pngtuber, streamer, stream, streaming
Horror	913	4.3%	horror, evil, resident, silent, hill, townfall, residentevil, movie
Indie dev	847	4.0%	gamedev, indiedev, indiegame, indiegames, steam, indiegamedev, pixelart, wishlist
AI & tech news	551	2.6%	ai, news, technology, art, music, photography, tv, business
General / mixed	12157	57.7%	art, music, youtube, making, anime, movies, free, story

```
ex <- tx_c %>% group_by(theme) %>% slice_sample(n = 2) %>%
  transmute(theme, example = str_trunc(str_squish(iconv(text, "UTF-8", "ASCII", sub = "")), 150))
kable(ex, caption = "Two random example posts from each theme.", booktabs = TRUE) %>%
  kableExtra::column_spec(2, width = "13cm") %>% kableExtra::kable_styling(latex_options = "hold_position", font_size
↪  = 8)
```

Visualising the clusters. We use **multidimensional scaling** (MDS, `cmdscale`) to place a random sample of 1,500 posts in 2-D so that their cosine distances are preserved as well as possible. We also plot the top terms of each cluster.

```
s2 <- sample(nrow(X), min(1500, nrow(X)))
mds <- cmdscale(dist(X[s2, ]), k = 2)
md <- data.frame(dim1 = mds[, 1], dim2 = mds[, 2], theme = tx_c$theme[s2])
p_mds <- ggplot(md, aes(dim1, dim2, colour = theme)) + geom_point(alpha = 0.6, size = 1.2) +
  scale_colour_manual(values = pal_k) + labs(x = "MDS dimension 1", y = "MDS dimension 2", colour = NULL) +
  theme(legend.position = "right")
```

Table 6: Two random example posts from each theme.

theme	example
Streamer promo	Mood: Gaming spaces feel stormy, with users frustrated by xQc stream timeouts and the grind of Pokemon shiny hunting. Political discourse is equall...
Streamer promo	It's time for our Demo Derby! www.twitch.tv/mcgurganatorzx #Twitch #Stream #DemoDerby
Go-live invites	LIVE NOW i get to put cbt in the stream title and it's probably relevant!! www.twitch.tv/possumgmr
Go-live invites	Live on Twitch! twitch.tv/arecarws Playing #Aion2 live on #twitchtv Come watch and chat! #livestreaming #sorcerer #angels #twitch #gaming #vtube #c...
PC & consoles	This upgrade lets you throw explosive daggers: lower damage on impact, but they detonate for splash damage once the timer runs out. Great for takin...
PC & consoles	The best-reviewed games on Steam released last week. TCG Card Shop Simulator - 9.5/10 20 More, Somehow Even Smaller, Mazes - 9.4/10 Emberward - 9.2...
Nintendo & retro	The Breath Of The Wilding" Makeover That Wasn't: Ocarina Of Time Redux: activetimeevent.com/2026/09/25/t... #gaming #design #philosophy #nintendo #...
Nintendo & retro	Man the worst thing you can really do in the eyes of some in retro gaming is "take too long on a free project" No wonder they're rallying behind & ...
VTubers	LIVE!! We are back playing the RP server twitch.tv/Glitchfox_ #Vtuber #Weird #smallstreamer #streamer #chillguy #affilate #funny #twitch #twitchstr...
VTubers	Live now Day 21 on Constellation! No plan tonight, so chat decides what the wolf builds. Hop on and join in, or just come hang out https://www.twit...
Horror	Ohhhh it's October! You know what that means! HORROR GAMES! In about an hour, I'm gonna start up a playthrough of Alan Wake, which uh - somehow I n...
Horror	I recently discovered Slender: The Arrival's devs promised a HUGE update and VR game 3+ years ago - what happened? I discuss the situation in this ...
Indie dev	Two days left for the steam sale. store.steampowered.com/bundle/51943... #IndieGameDev #Steam #demo #indiegame #indiedev #games #playtest #videogam...
Indie dev	David Slade: Fallen City. Question - I am curious what do you think of the ascetics of my game, I am personally happy with the end result. A dark &...
AI & tech news	Like if they can't even do this, I doubt their announcement about making purely AI video games will be any better. More janky and buggy than even t...
AI & tech news	Toheen-e-Risalat Ke Qawaneen | ALRA TV. #alratv #Ifollowgoharshahi #AI #digitalart #photography #nowplaying #music #gaming #news #booksky #Goharsha...
General / mixed	Are you a UK-based Gaming Creator? Did you about @safeinourworld.org Employee Assistance Program? It cover "a comprehensive range of support and se...
General / mixed	omgggg runescape, what a throwback ive not beeing playing much recently except for stardew valley tbh, but im thinking about getting a gaming lapto...

```
p_mds
```

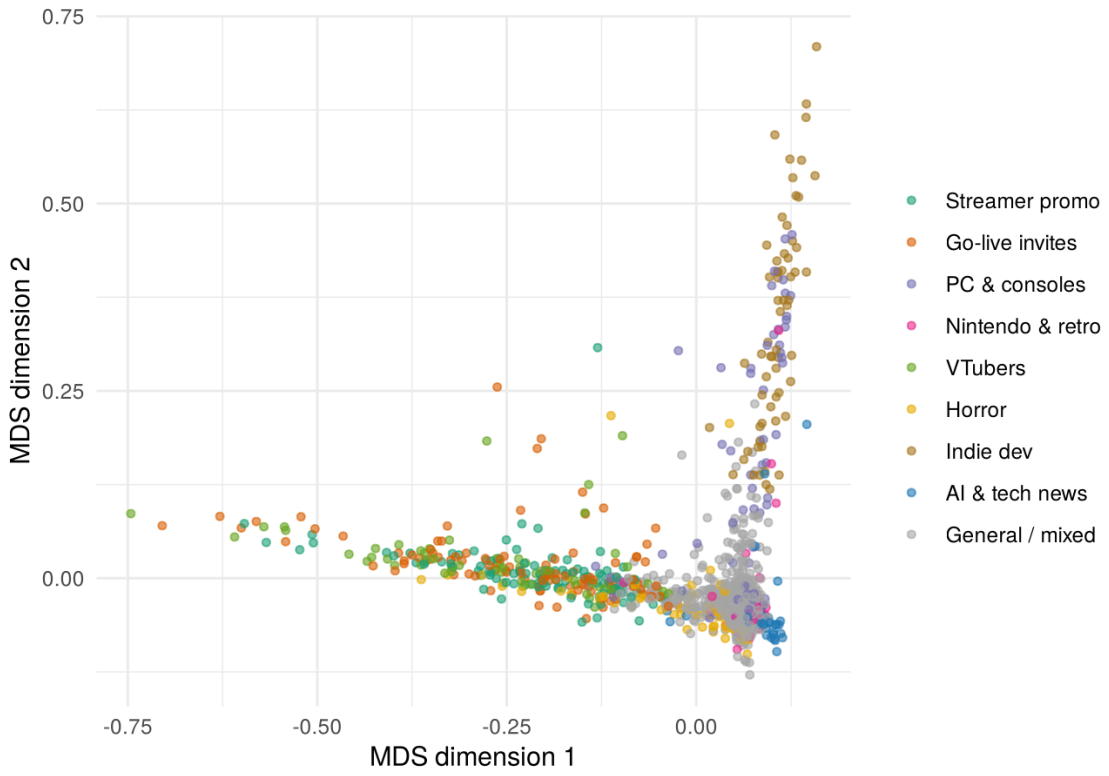


Figure 7: Classical MDS of 1,500 sampled posts (cosine distance), coloured by k-means cluster.

```
ggsave("figures/fig_mds.png", p_mds, width = 7, height = 4.5, dpi = 300)
```

```
ct <- do.call(rbind, lapply(1:K, function(i) {
  v <- sort(km$centers[i, ], decreasing = TRUE)[1:6]
  data.frame(theme = theme_names[i], term = names(v), weight = as.numeric(v))}))
ct$theme <- factor(ct$theme, levels = levels(tx_c$theme))
p_ct <- ggplot(ct, aes(reorder(paste(term, theme, sep = "___"), weight), weight, fill = theme)) +
  geom_col(show.legend = FALSE) + coord_flip() + facet_wrap(~theme, scales = "free_y") +
  scale_x_discrete(labels = function(x) sub("___.*$", "", x)) + scale_fill_manual(values = pal_k) +
  labs(x = NULL, y = "Mean TF-IDF weight in cluster centroid")
p_ct
```

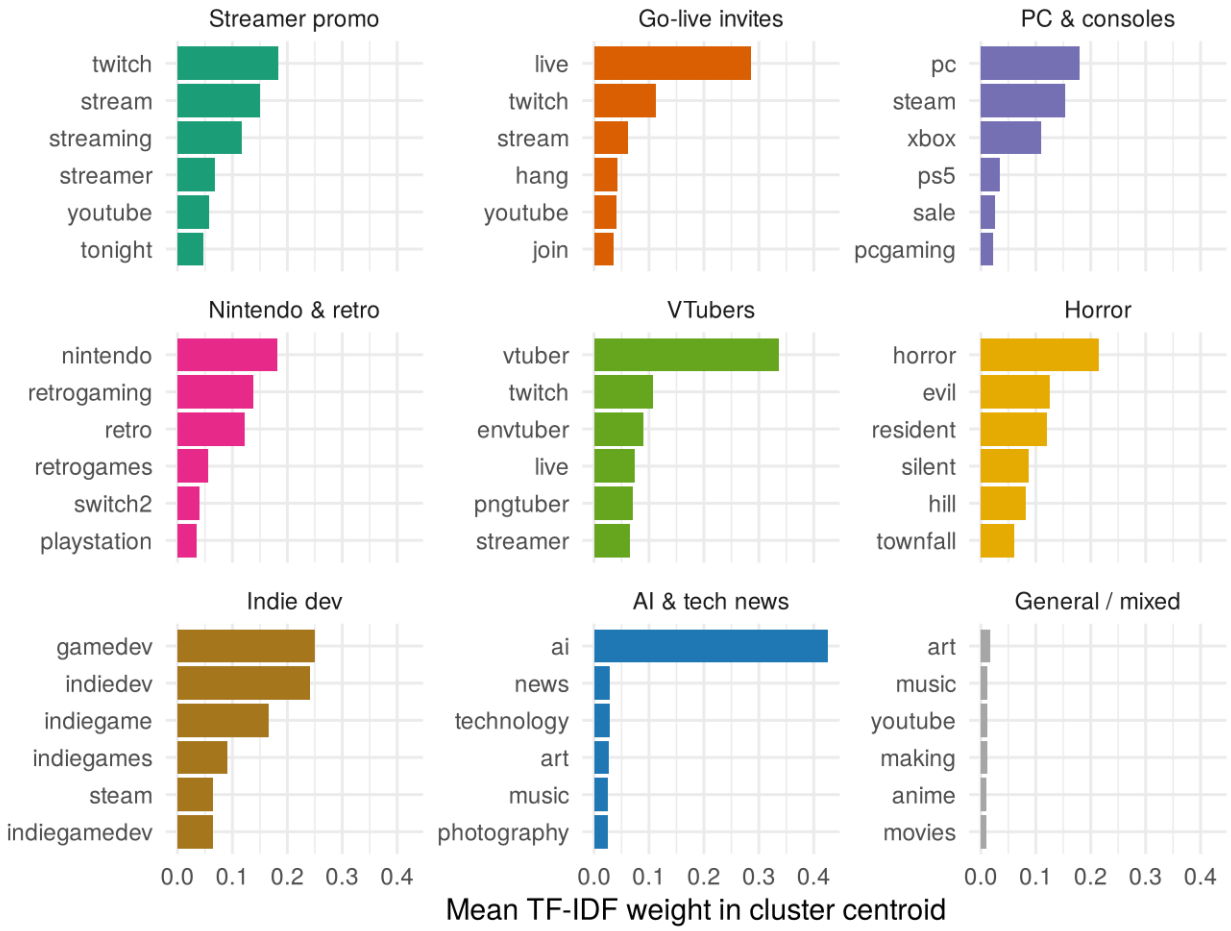


Figure 8: Top 6 terms in each cluster centroid.

```
ggsave("figures/fig_cluster_terms.png", p_ct, width = 9, height = 6, dpi = 300)
```

Is the clustering robust? To check that the themes are not an artefact of k-means, we cluster the same 1,500 sampled posts with **hierarchical clustering** (Ward's method on the unit-length vectors, i.e. cosine geometry). We cut the tree into K groups and compare it with the k-means labels. *Purity* is the share of posts whose hierarchical group's majority k-means cluster matches their own k-means cluster (1 = perfect agreement).

```
hc <- hclust(dist(X[s2, ]), method = "ward.D2")
hc_lab <- cutree(hc, K)
tab_hk <- table(hierarchical = hc_lab, kmeans = theme_names[km$cluster[s2]])
purity <- sum(apply(tab_hk, 1, max)) / sum(tab_hk)
baseline <- max(table(km$cluster[s2])) / length(s2)    # purity if hclust put everything in one group
kable(as.data.frame.matrix(t(tab_hk)), booktabs = TRUE, longtable = FALSE,
      caption = "k-means theme (rows) against hierarchical group 1-9 (columns), 1,500 sampled posts.") %>%
  kableExtra::kable_styling(latex_options = "hold_position")
```

```
c(purity = round(purity, 3), baseline_largest_cluster = round(baseline, 3))
```

```
#>                  purity baseline_largest_cluster
#>                   0.643                   0.597
```

Interpretation (RQ2). Yes, but only part of the conversation forms clear themes. Eight distinct themes contain 42% of the clustered posts. The rest (58%) is a **general/mixed** cluster of everyday chat with no dominant vocabulary. **Streaming is the largest identifiable activity.** Three themes are about streaming and together hold 19% of posts. They differ in style: *Streamer promo* (channel promotion, Discord), *Go-live invites* ("live now, come hang out") and *VTubers* (identity tags such as #envtuber and #pngtuber). The other themes match RQ1:

Table 7: k-means theme (rows) against hierarchical group 1-9 (columns), 1,500 sampled posts.

	1	2	3	4	5	6	7	8	9
AI & tech news	39	0	5	0	0	0	0	0	0
General / mixed	709	12	125	19	22	2	0	2	5
Go-live invites	19	0	55	27	4	0	0	1	2
Horror	33	0	3	4	0	14	14	0	3
Indie dev	16	41	5	1	0	0	0	0	0
Nintendo & retro	55	1	4	0	6	0	0	0	0
PC & consoles	61	15	9	1	1	0	0	6	0
Streamer promo	27	1	37	29	1	0	0	0	1
VTubers	3	0	28	28	2	0	0	0	2

Indie dev (wishlists, demos, pixel art), *PC & consoles* (hardware and Steam sales), *Nintendo & retro*, a seasonal *Horror* theme (*Resident Evil*, *Silent Hill*) and *AI & tech news*.

The MDS plot shows the themed clusters spreading out from a dense centre of general posts, so the boundaries are fuzzy. This agrees with the low silhouette. The hierarchical check gives a purity of 0.64, only a little above the 0.6 we would get by putting everything into the largest cluster. Reading the table, however, Ward clustering also separates **Indie dev**, the **streaming family** (VTubers, Go-live, Streamer promo) and **Horror** into their own groups, but folds Nintendo, PC & consoles and AI into its large general group. We therefore treat the indie, streaming and horror themes as robust, and the other themes as useful but method-dependent. These themes are the categories we use to test engagement in RQ3.

6 RQ3: Does the theme of a post affect its engagement?

The clusters suggest the gaming conversation contains very different kinds of content. Is the community equally responsive to all of them? We measure engagement as whether a post received **at least one like**. A binary outcome is robust to the extremely skewed like counts, where a few viral posts have thousands of likes and most have none. We make two decisions to keep the comparison fair:

- **Only original posts (not replies).** Replies are seen mainly by thread participants and get far fewer likes, and some themes contain more replies than others.
- **Exclude posts written in the 24 hours before collection.** They have not had a full day to collect likes.

```
hd <- tx_c %>% filter(!is_reply, created < collected - 24 * 3600) %>%
  mutate(liked = ifelse(like_count >= 1, "Liked (>=1)", "No likes"))
nrow(hd)
```

```
#> [1] 16296
```

6.1 Test 1: Engagement versus theme (chi-squared test of independence)

- H_0: getting at least one like is **independent** of a post's theme (every theme has the same probability of being liked).
- H_1: getting a like **depends** on theme.

The chi-squared test of independence is appropriate because both variables are categorical (theme with 9 levels; liked yes/no), and the observations are counts of posts in a contingency table. It requires all expected counts to be at least 5 (checked below).

```
tab <- table(hd$theme, hd$liked)
ct_res <- chisq.test(tab)
ct_res
```

```
#>
#>  Pearson's Chi-squared test
```

```
#>
#> data:  tab
#> X-squared = 583.33, df = 8, p-value < 2.2e-16
```

```
min(ct_res$expected)                    # assumption check: all expected counts >= 5
```

```
#> [1] 64.5764
```

```
lk <- hd %>% group_by(theme) %>% summarise(n = n(), liked = mean(like_count >= 1)) %>%
  mutate(stdres = ct_res$stdres[as.character(theme), "Liked (>=1)"])
p_lk <- ggplot(lk, aes(reorder(theme, liked), liked, fill = theme)) + geom_col(show.legend = FALSE) +
  geom_hline(yintercept = mean(hd$like_count >= 1), linetype = 2) +
  geom_text(aes(label = sprintf("z = %+.1f", stdres)), hjust = -0.1, size = 3.2) +
  coord_flip() + scale_y_continuous(labels = percent, expand = expansion(mult = c(0, 0.25))) +
  scale_fill_manual(values = pal_k) + labs(x = NULL, y = "Share of posts with at least one like")
p_lk
```

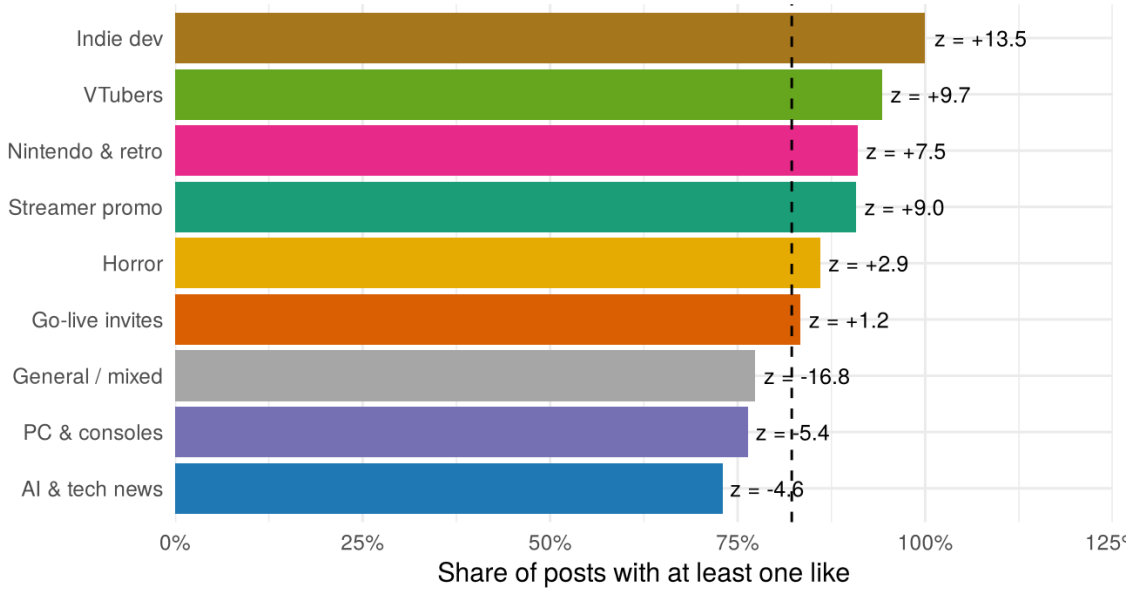


Figure 9: Share of original posts receiving at least one like, by theme (dashed line = overall share). Labels show standardised residuals from the chi-squared test; |residual| > 2 indicates a theme that differs clearly from independence.

```
ggsave("figures/fig_engagement_theme.png", p_lk, width = 7, height = 4, dpi = 300)
```

Interpretation (Test 1). $\chi^2 = 583.3$, df = 8, $p < 0.001$, and all expected counts are at least 5 (minimum 65). We **reject** H_0: whether a gaming post gets liked depends on its theme. The standardised residuals show the direction. **Community/creator themes** are liked more often than independence predicts: Indie dev (100% liked), VTubers (94%), Streamer promo (91%) and Nintendo & retro (91%). **Information themes** are liked less often: General (77%), PC & consoles (76%) and AI & tech news (73%).

An anomaly: the indie "like floor". Every Indie dev post received at least one like. The table below shows that indie posts *never* receive fewer than 3 likes, while every other theme has many posts with 0 or 1. Looking across *all* original posts carrying #indiedev, #gamedev or #indiegamedev, every one of them was liked, and they came from hundreds of different accounts. We cannot see *who* liked them, but such a regular floor strongly suggests **automated accounts that like every post with these hashtags**. So likes overstate genuine human attention for indie developers. This is an important caveat for anyone using likes as a measure of popularity on Bluesky.

```
kable(hd %>% group_by(theme) %>% summarise(posts = n(), `min likes` = min(like_count),
      `% with 0 likes` = percent(mean(like_count == 0), 0.1), `% with exactly 1` = percent(mean(like_count == 1),
      ↪  0.1),
      `median likes` = median(like_count), `median followers` = median(followers, na.rm = TRUE)),
```

```
    caption = "Like distribution and audience size by theme.", booktabs = TRUE)
```

Table 8: Like distribution and audience size by theme.

theme	posts	min likes	% with 0 likes	% with exactly 1	median likes	median followers
Streamer promo	1449	0	9.2%	12.4%	3	104.0
Go-live invites	1416	0	16.6%	16.9%	2	98.0
PC & consoles	1162	0	23.6%	15.5%	2	172.0
Nintendo & retro	1001	0	9.0%	9.6%	4	196.0
VTubers	896	0	5.7%	13.6%	3	120.5
Horror	795	0	14.0%	15.7%	3	174.0
Indie dev	801	3	0.0%	0.0%	12	99.0
AI & tech news	363	0	27.0%	18.2%	2	243.0
General / mixed	8413	0	22.7%	16.8%	2	239.0

```
# all original posts (not just clustered ones) carrying an indie-dev hashtag
posts %>% filter(!is_reply, created < collected - 24 * 3600) %>%
  mutate(indie_hashtag = grepl("(^|;)(indiedev|gamedev|indiegamedev)(;|$)", hashtags)) %>%
  group_by(indie_hashtag) %>% summarise(posts = n(), accounts = n_distinct(author_did),
  share_liked = mean(like_count >= 1), min_likes = min(like_count))
```

```
#> # A tibble: 2 x 5
#>   indie_hashtag posts accounts share_liked min_likes
#>   <lgl>         <int>    <int>       <dbl>     <int>
#> 1 FALSE         33167    14578       0.750         0
#> 2 TRUE           1487      417       1             1
```

Sensitivity check. Because of this anomaly, we repeated the test **without the Indie dev theme**:

```
h2 <- droplevels(hd %>% filter(theme != "Indie dev"))
ct2 <- chisq.test(table(h2$theme, h2$liked)); ct2
```

```
#>
#>  Pearson's Chi-squared test
#>
#> data:  table(h2$theme, h2$liked)
#> X-squared = 385.64, df = 7, p-value < 2.2e-16
```

```
round(ct2$stdres[, "Liked (>=1)"], 1)
```

```
#>   Streamer promo  Go-live invites    PC & consoles Nintendo & retro          VTubers
#>              9.8             2.1             -4.4              8.2             10.3
#>           Horror   AI & tech news  General / mixed
#>              3.5            -4.1            -13.8
```

The result is unchanged ($\chi^2 = 385.6$, df $= 7$, $p < 0.001$), with the same pattern of residuals. So the dependence on theme is not just an artefact of the indie like-bots. **Audience size does not explain it either.** The table shows that the *less*-liked themes are posted by accounts with *more* followers (median 239 for General and 243 for AI & tech news, against 104 for Streamer promo). Our interpretation is that streamers, VTubers and indie developers form **reciprocal support communities**, where creators routinely like each other's posts (the "#smallstreamer" culture). News, hardware and AI posts are read but rarely liked.

6.2 Test 2: Do posts with images or video get more likes? (Welch two-sample t-test)

- $H_0 : \mu_{\text{media}} = \mu_{\text{no media}}$, where μ is the mean of $\log(1 + \text{likes})$
- $H_1 : \mu_{\text{media}} \neq \mu_{\text{no media}}$

Raw like counts are extremely right-skewed. We therefore compare the means of $\log(1 + \text{likes})$, which reduces the influence of a few viral posts. The samples are large, so by the Central Limit Theorem the sample means are approximately normal even though the data are not. We use **Welch's** version because the two groups need not have equal variances.

```
hd$log_likes <- log1p(hd$like_count)
tt <- t.test(log_likes ~ has_media, data = hd)          # Welch is R's default
tt
```

```
#>
#>  Welch Two Sample t-test
#>
#> data:  log_likes by has_media
#> t = -25.977, df = 12381, p-value < 2.2e-16
#> alternative hypothesis: true difference in means between group FALSE and group TRUE is not equal to 0
#> 95 percent confidence interval:
#>  -0.4643011 -0.3991473
#> sample estimates:
#> mean in group FALSE  mean in group TRUE
#>            1.231153             1.662877
```

```
hd %>% group_by(has_media) %>% summarise(posts = n(), mean_log_likes = mean(log_likes),
                                         median_likes = median(like_count), share_liked = mean(like_count >= 1))
```

```
#> # A tibble: 2 x 5
#>   has_media posts mean_log_likes median_likes share_liked
#>   <lgl>     <int>          <dbl>        <dbl>       <dbl>
#> 1 FALSE     10066           1.23            2       0.786
#> 2 TRUE       6230           1.66            4       0.881
```

```
hd$indie_tag <- grepl("(^|;)(indiedev|gamedev|indiegamedev)(;|$)", hd$hashtags)
tt2 <- t.test(log_likes ~ has_media, data = hd %>% filter(!indie_tag)); tt2$statistic; tt2$conf.int
```

```
#>         t
#> -20.48211
```

```
#> [1] -0.3853307 -0.3180186
#> attr(,"conf.level")
#> [1] 0.95
```

Interpretation (Test 2). t = -26.0, df $\approx$ 12,381, $p < 0.001$. We **reject** H_0: posts with images or video get more likes. The mean of $\log(1 + \text{likes})$ is 1.66 with media against 1.23 without. That difference of 0.43 (95% CI 0.4 to 0.46) means $(1 + \text{likes})$ is about **1.54 times higher** for media posts (a geometric-mean comparison). The median doubles from 2 to 4 likes, and the share of posts liked rises from 79% to 88%. Indie posts often include screenshots and get bot likes, so we repeated the test without them. The effect remains (t = -20.5). This is an association, not proof that adding an image *causes* more likes: creators who attach media may also put more effort into their posts in other ways.

7 RQ4: Who connects the gaming conversation? (network analysis)

Nodes and edges. Nodes are **Bluesky accounts**. A directed edge $A \to B$ means that, in a collected gaming post, account A **replied to**, **mentioned** or **quoted** account B. All three are recorded in the post data itself: the reply's parent URI contains the parent author's ID, mention facets contain the mentioned account's ID, and quote embeds contain the quoted post's URI. The edges are therefore real interactions, not constructed similarities. The graph is **directed** because the interactions are one-way: A giving attention to B does not mean B gave attention to A. Repeated interactions are stored as edge **weights**. Self-loops (people replying to their own threads) are removed, because they do not connect different accounts. We use *all* collected posts in the window (before the text-cleaning filters), because a reply such as "agreed!" is still a real interaction even if it is too short for text analysis.

```r
dup_bot <- duplicated(posts$clean)          # still drop exact-duplicate spam posts
pn <- posts[!dup_bot, ]
e_rep <- pn %>% filter(!is.na(reply_to_did)) %>% transmute(from = author_did, to = reply_to_did, type = "reply")
e_quo <- pn %>% filter(!is.na(quoted_did)) %>% transmute(from = author_did, to = quoted_did, type = "quote")
e_men <- pn %>% filter(mention_dids != "") %>% select(author_did, mention_dids) %>%
  separate_rows(mention_dids, sep = ";") %>% transmute(from = author_did, to = mention_dids, type = "mention")
edges <- bind_rows(e_rep, e_quo, e_men) %>% filter(from != to)
table(edges$type)
```

```
#>
#> mention   quote   reply
#>    2452    2139    6249
```

```r
ew <- edges %>% count(from, to, name = "weight")
g <- graph_from_data_frame(ew, directed = TRUE)
# node attributes: handle, followers, dominant theme of the account's clustered posts
V(g)$handle <- profiles$handle[match(V(g)$name, profiles$did)]
V(g)$handle[is.na(V(g)$handle)] <- substr(V(g)$name[is.na(V(g)$handle)], 9, 16)
V(g)$followers <- profiles$followers[match(V(g)$name, profiles$did)]
dom <- tx_c %>% count(author_did, theme) %>% group_by(author_did) %>% slice_max(n, n = 1, with_ties = FALSE)
V(g)$theme <- as.character(dom$theme[match(V(g)$name, dom$author_did)])
V(g)$theme[is.na(V(g)$theme)] <- "No clustered posts"
```

```r
comp <- components(g, mode = "weak")
net_stats <- data.frame(
  Measure = c("Nodes (accounts)", "Edges (distinct directed pairs)", "Total interactions (weighted)",
              "Density", "Weakly connected components", "Largest component: nodes",
              "Largest component: share of nodes", "Reciprocity", "Mean in-degree", "Max in-degree"),
  Value = c(vcount(g), ecount(g), sum(E(g)$weight), signif(edge_density(g), 3), comp$no,
            max(comp$csize), percent(max(comp$csize) / vcount(g), 0.1), round(reciprocity(g), 3),
            round(mean(degree(g, mode = "in")), 2), max(degree(g, mode = "in"))))
kable(net_stats, caption = "Network summary.", booktabs = TRUE)
```

Table 9: Network summary.

Measure	Value
Nodes (accounts)	14202
Edges (distinct directed pairs)	9832
Total interactions (weighted)	10840
Density	4.87e-05
Weakly connected components	4640
Largest component: nodes	2489
Largest component: share of nodes	17.5%
Reciprocity	0.031
Mean in-degree	0.69
Max in-degree	341

```r
dd <- data.frame(k = degree(g, mode = "in")) %>% filter(k > 0) %>% count(k)
ggplot(dd, aes(k, n)) + geom_point(colour = "#0072B2") + scale_x_log10() + scale_y_log10() +
  labs(x = "In-degree (number of distinct accounts interacting with node)", y = "Number of accounts")
```

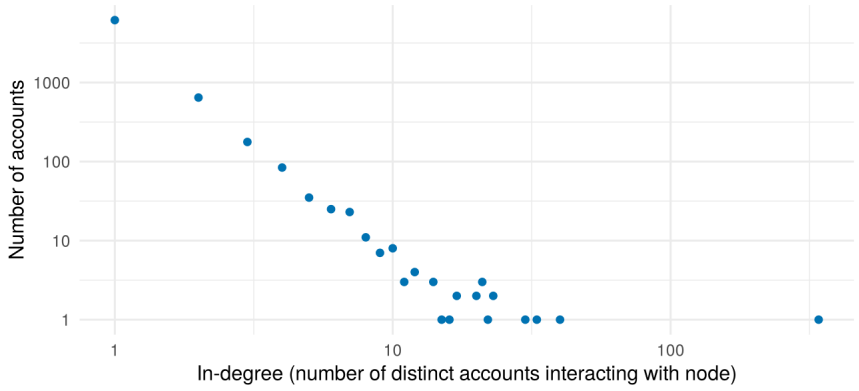


Figure 10: In-degree distribution on log-log axes. An approximately straight line indicates a heavy-tailed distribution: most accounts receive one interaction, and a few receive very many.

Visualising the full network. Nodes are coloured by the account's dominant theme from RQ2. Grey nodes are accounts that were only *interacted with* (or whose posts were filtered out) and so have no theme. Node size reflects in-degree.

```
node_pal <- c(pal_k, `No clustered posts` = "grey80")
# Each component is laid out separately (Fruchterman-Reingold) and the components are packed
# together; otherwise tiny components get pushed to the edges and the picture is unreadable.
lay_full <- layout_components(g, layout = layout_with_fr)   # computed once, reused for the PNG
plot_full <- function() {
  par(mar = c(0, 0, 0, 0))
  plot(g, layout = lay_full, vertex.label = NA, edge.arrow.size = 0, edge.width = 0.2,
       edge.color = adjustcolor("grey50", 0.3), vertex.frame.color = NA,
       vertex.color = node_pal[V(g)$theme], vertex.size = 0.8 + 2.5 * sqrt(degree(g, mode = "in")) / 4)
  legend("bottomleft", legend = names(node_pal), fill = node_pal, cex = 0.6, bty = "n")
}
plot_full()
```



Figure 11: The full interaction network (Fruchterman-Reingold layout per component). Colour = dominant theme of the account; size = in-degree.

```
png("figures/fig_network_full.png", width = 2400, height = 2400, res = 300); plot_full(); invisible(dev.off())
```

Centrality. We calculate three measures on the full directed network:

- **In-degree**: the number of distinct accounts that replied to, mentioned or quoted the node. It measures *attention received*.
- **PageRank** (weighted): an account is important if it receives interactions from other important accounts. It measures *influence*, discounting attention that comes only from peripheral accounts.
- **Betweenness**: how often the node lies on shortest paths between other accounts. It measures *bridging*, i.e. accounts that connect otherwise separate parts of the conversation.

```r
cent <- data.frame(handle = V(g)$handle, theme = V(g)$theme, followers = V(g)$followers,
                   in_degree = degree(g, mode = "in"), out_degree = degree(g, mode = "out"),
                   pagerank = page_rank(g, weights = E(g)$weight)$vector,
                   betweenness = betweenness(g, directed = TRUE, weights = NA))
top_tab <- function(col, n = 8) cent %>% arrange(desc(.data[[col]])) %>% head(n) %>%
  transmute(handle, theme, followers, in_degree, pagerank = signif(pagerank, 3), betweenness = round(betweenness))
kable(top_tab("in_degree"), caption = "Top accounts by in-degree.", booktabs = TRUE, row.names = FALSE)
```

Table 10: Top accounts by in-degree.

handle	theme	followers	in_degree	pagerank	betweenness
retroblastpod.com	Nintendo & retro	37749	341	0.012400	6252
airbagged.bsky.social	Streamer promo	46593	40	0.002220	131
wario64.bsky.social	General / mixed	154755	33	0.001100	0
ign.com	No clustered posts	210763	30	0.001050	0
aftermath.site	No clustered posts	132294	23	0.000873	0
bugsmaytrix.bsky.social	No clustered posts	36294	23	0.001080	0
rincewind.run	No clustered posts	103575	22	0.000880	0
atrupar.com	No clustered posts	974640	21	0.000867	0

```r
kable(top_tab("pagerank"), caption = "Top accounts by PageRank.", booktabs = TRUE, row.names = FALSE)
```

Table 11: Top accounts by PageRank.

handle	theme	followers	in_degree	pagerank	betweenness
retroblastpod.com	Nintendo & retro	37749	341	0.01240	6252
ryanwritesgood.bsky.social	General / mixed	1119	2	0.00375	4756
gtothenextlevel.com	No clustered posts	1551	1	0.00355	393
spacenacho.bsky.social	No clustered posts	782	1	0.00355	0
evercade.co.uk	No clustered posts	4242	3	0.00311	0
airbagged.bsky.social	Streamer promo	46593	40	0.00222	131
beardyjim.com	PC & consoles	1967	10	0.00138	1668
sophianyx.bsky.social	General / mixed	23494	21	0.00127	0

```r
kable(top_tab("betweenness"), caption = "Top accounts by betweenness.", booktabs = TRUE, row.names = FALSE)
```

Table 12: Top accounts by betweenness.

handle	theme	followers	in_degree	pagerank	betweenness
retroblastpod.com	Nintendo & retro	37749	341	0.012400	6252
ryanwritesgood.bsky.social	General / mixed	1119	2	0.003750	4756
beardyjim.com	PC & consoles	1967	10	0.001380	1668
andresplays.bsky.social	General / mixed	9456	8	0.000758	814
jtp.bsky.social	AI & tech news	98352	8	0.000723	617
gtothenextlevel.com	No clustered posts	1551	1	0.003550	393
freeuse.toys	No clustered posts	6494	6	0.000349	350
thedimpsler.bsky.social	General / mixed	1532	1	0.000104	266

```r
cor(cent[, c("in_degree", "pagerank", "betweenness", "followers")], method = "spearman", use = "pairwise")
```

```
#>              in_degree  pagerank betweenness followers
#> in_degree    1.0000000 0.9538894   0.2782793 0.3212314
#> pagerank     0.9538894 1.0000000   0.2764606 0.3178668
#> betweenness 0.2782793 0.2764606   1.0000000 0.1140803
#> followers    0.3212314 0.3178668   0.1140803 1.0000000
```

Subgraph: the core of the conversation. The full graph is dominated by many small, star-shaped groups. To see who sits in the densely connected *core*, we take the largest weakly connected component and keep its **k-core**: the largest subgraph in which every account has at least k connections to other accounts in the subgraph. We choose the largest k that still leaves at least 40 accounts, which is enough for a readable but meaningful picture.

```r
lcc <- induced_subgraph(g, which(comp$membership == which.max(comp$csize)))
to_undirected <- if (exists("as_undirected", asNamespace("igraph"))) igraph::as_undirected else igraph::as.undirected
core <- coreness(to_undirected(lcc, mode = "collapse"))
k_sel <- max(which(sapply(1:max(core), function(k) sum(core >= k)) >= 40))
sub <- induced_subgraph(lcc, which(core >= k_sel))
pr_sub <- page_rank(sub, weights = E(sub)$weight)$vector
lab <- ifelse(rank(-pr_sub, ties.method = "first") <= 15, V(sub)$handle, NA)
lay_sub <- norm_coords(layout_with_kk(sub))   # Kamada-Kawai spreads the dense hub better than FR
plot_sub <- function(lab_cex = 0.6) {
  par(mar = c(0, 0, 0, 0))
  plot(sub, layout = lay_sub, rescale = FALSE, xlim = c(-1, 1), ylim = c(-1, 1), vertex.label = NA,
       vertex.color = node_pal[V(sub)$theme], vertex.frame.color = "white",
       vertex.size = 3 + 40 * sqrt(pr_sub), edge.arrow.size = 0.2, edge.width = 0.4,
       edge.color = adjustcolor("grey40", 0.4))
  li <- which(!is.na(lab))                # wordcloud::textplot places labels without overlaps
  wordcloud::textplot(lay_sub[li, 1], lay_sub[li, 2], sub("\\.bsky\\.social$", "", lab[li]),
                      new = FALSE, cex = lab_cex, font = 2, xlim = c(-1, 1), ylim = c(-1, 1))
  legend("bottomleft", legend = unique(V(sub)$theme), fill = node_pal[unique(V(sub)$theme)], cex = 0.6, bty = "n")
}
plot_sub()
```

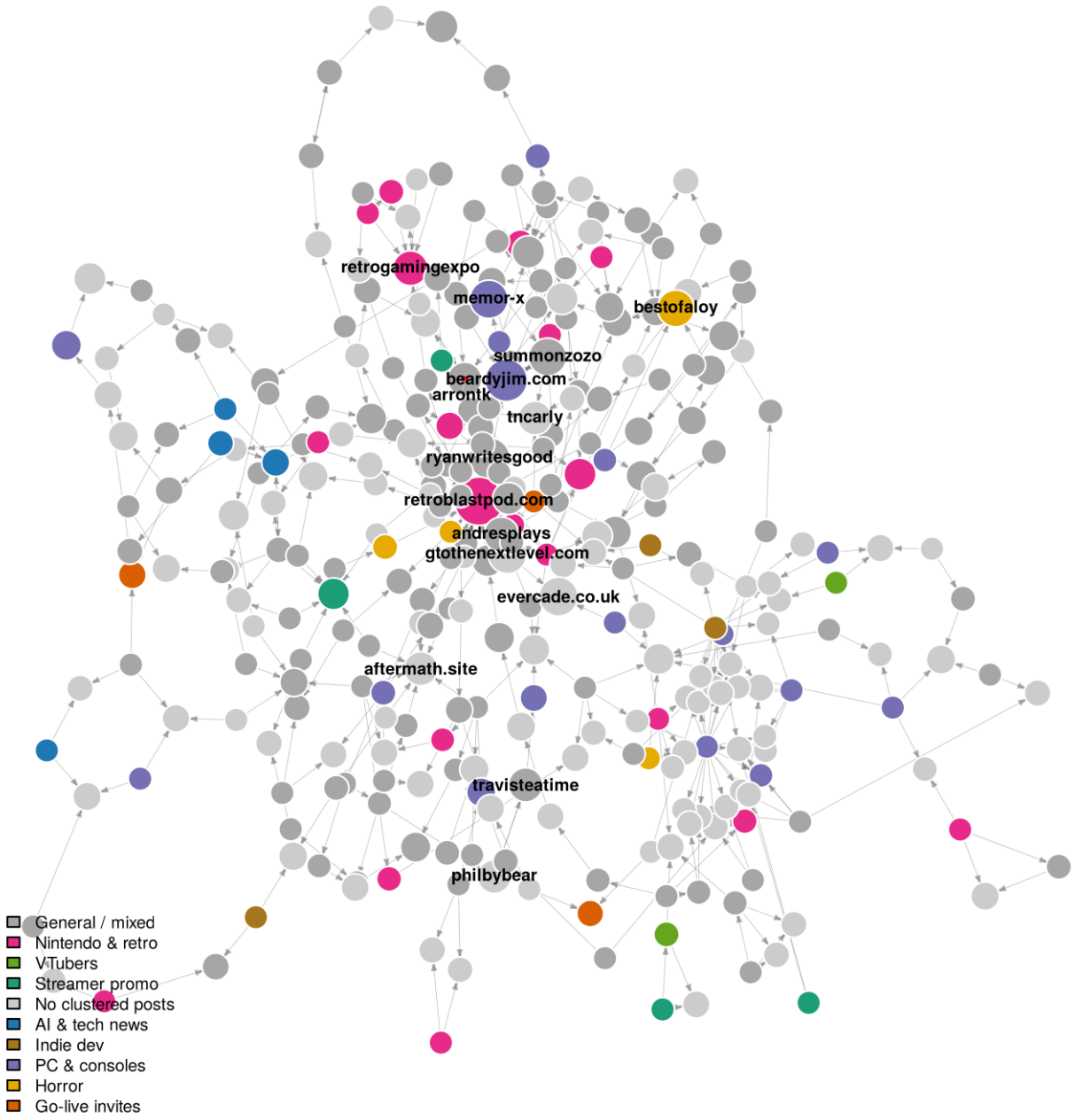


Figure 12: The k-core of the largest component (Kamada-Kawai layout). Colour = dominant theme; size = PageRank; labels = top 15 accounts by PageRank.

```
png("figures/fig_network_core.png", width = 2400, height = 2400, res = 300); plot_sub(); invisible(dev.off())
c(k = k_sel, nodes = vcount(sub), edges = ecount(sub))
```

```
#>     k nodes edges
#>     2   313   429
```

Do accounts interact within their own theme? We compute the **nominal assortativity** coefficient of the network with respect to theme, using only accounts that have a theme. It ranges from -1 (interactions always cross themes), through 0 (theme makes no difference to who interacts with whom), to $+1$ (interactions only happen within themes).

```r
g_th <- induced_subgraph(g, V(g)$theme != "No clustered posts")
assort <- assortativity_nominal(g_th, types = as.integer(factor(V(g_th)$theme)), directed = TRUE)
c(accounts_with_theme = vcount(g_th), edges_between_them = ecount(g_th), assortativity = round(assort, 3))
```

```
#> accounts_with_theme  edges_between_them       assortativity
#>            4494.000            1358.000               0.103
```

Interpretation (RQ4). *Structure.* The network is **extremely sparse and fragmented**. Its density is 4.9×10^{-5}, it has 4640 separate components, and the largest component holds only 17.5% of accounts. Only 3.1% of ties are reciprocated, and the in-degree distribution is heavy-tailed. In other words, gaming Bluesky is mostly **broadcasting**: people post *at* accounts far more than they talk *with* each other. Most of the 9708 grey nodes are accounts that were only replied to, mentioned or quoted.

Important nodes. The top hub, **retroblastpod.com** (in-degree 341, more than eight times the next account), is explained by **one prompt post**: *"What's your origin video game?"*. It received about 300 quote posts, almost all of them answers to that question. This is the same nostalgic "games that shaped me" behaviour behind the #my9games trend in RQ1. Large news and personality accounts (**ign.com**, **aftermath.site**, **wario64**) appear in the in-degree top 10, but their **betweenness is 0**. They are *endpoints* that people point at, not bridges. The bridges are mid-sized personal accounts such as **ryanwritesgood**, **beardyjim.com** and **andresplays**, which link the retro hub's conversation to other groups. **PageRank** shows how importance flows. Accounts such as *gtothenextlevel.com* and *spacenacho* rank highly despite an in-degree of 1, because their single in-link comes from the hub itself. Follower counts are only weakly related to in-degree (Spearman $\rho = 0.32$), so **being popular is not the same as being central** in the conversation.

Core and themes. The 2-core of the largest component (313 accounts) is centred on the **retro-gaming community**: retroblastpod.com, retrogamingexpo (PRGE, which was trending in RQ1) and evercade.co.uk. Nintendo & retro and PC & consoles are the most common themes there, after General. Assortativity by theme is 0.1, a **weak positive** value. Accounts interact slightly more within their own theme than chance, but mostly the themes are *styles of content* rather than separate communities.

How data collection affects the network. The network only contains interactions that happened *inside posts matching our gaming search terms*. A reply that does not itself mention "gaming" is invisible, so threads are cut short and the graph is more fragmented than the real conversation. Reposts and likes cannot be collected through search, and follows were not collected, so the network captures *conversational* attention only. The 100-posts-per-block cap under-samples busy periods, which may remove interactions around big events. Accounts that are only interacted with have no posts in our sample, so they have no theme (grey). Finally, centrality is computed on a sample, so the rankings describe *this sample's* gaming conversation, not Bluesky as a whole.

8 Overall findings

Putting the four analyses together, **what's trending in gaming on Bluesky is creators, communities and nostalgia rather than blockbuster news.**

1. **Creators dominate the volume (RQ1, RQ2).** The most frequent vocabulary and the largest identifiable themes are live-streaming (three themes, nearly half of all themed posts) and indie development. Platform news, franchises, horror and AI make up smaller themes, and more than half of the posts are general chat.
2. **Trends are event-driven and short-lived (RQ1).** Over four weeks only a handful of words changed significantly. They were tied to an event (Portland Retro Gaming Expo), a release (*Gears of War: E-Day*), a meme (#my9games) and a growing debate (AI).
3. **Communities reward each other (RQ3).** Streamer, VTuber, indie and Nintendo/retro posts are significantly more likely to be liked than news, hardware and AI posts, even though the latter come from accounts with more followers. Images and video are associated with roughly 1.5 times more likes. However, likes on indie posts show a floor that points to automated liking, a warning against treating likes as human approval.
4. **Conversation is sparse, and nostalgia connects it (RQ4).** The interaction network is broadcast-like and fragmented. Its biggest hub and its densest core come from the retro community and a single nostalgic prompt. Big outlets receive attention but do not bridge communities.

Each stage informed the next. The frequent words (RQ1) suggested themes, which clustering formalised (RQ2). Those themes became the variable in our engagement test (RQ3), and the test's anomaly (bot likes) and the trending

nostalgia meme helped explain the network's hubs (RQ4).

9 Limitations

- **Keyword sampling:** only posts that explicitly say "gaming"/"video games"/"gamer" are captured. This probably over-represents self-promotion (developers and streamers tag their posts) and under-represents fan discussion of specific titles.
- **Sampling caps and time window:** 100 posts per term per 6-hour block, over four weeks. Trends are relative to this window only.
- **Engagement snapshot and automation:** likes were measured once. Despite excluding the final 24 hours, older posts had more time to accumulate likes. Likes can also come from bots (see the indie "like floor"), so they are an imperfect measure of human attention. We could not check *who* liked posts without further API calls (`app.bsky.feed.getLikes`).
- **Confounding:** the chi-squared and t-tests show *association*, not causation. Follower count, posting time and account type may explain part of the differences.
- **Independence assumption:** several posts by the same account are not independent observations. The per-account cap reduces, but does not remove, this.
- **Clustering choices:** results depend on stop words, the sparsity threshold, TF-IDF weighting and k. Short posts (max 300 characters) make clusters noisy.
- **Network construction:** only conversational edges (reply/mention/quote) visible in matched posts are included. Reposts, likes and follows are missing. Centrality is sensitive to single events: one prompt post made retroblastpod.com the dominant hub.
- **Language tags:** some posts tagged English were partly Portuguese or Spanish (e.g. Brazilian promotional accounts), which adds noise to the text analysis.

10 Reproducibility

All code and data are in the GitHub repository linked on the cover page. Run `R/01_collect_bluesky.R` to collect new data (needs a Bluesky app password, which is never stored in the repository), then knit `report.Rmd`. Re-collecting will give different posts because Bluesky changes constantly; the exact data used in this report are in `data/`.

```
#> R version 4.3.3 (2024-02-29)
#> packages: tm 0.7.11, igraph 1.6.0, cluster 2.1.6, ggplot2 3.4.4, dplyr 1.1.4, wordcloud 2.6
```